Pipeline memory benchmark

Component costs, camera-count scaling, Docker overhead and unbounded-queue behavior on one Windows machine.

200 MiB
4-camera fast-consumer peak

250 MiB
App-only allowance, +25%

93.6
MiB/min queue growth when slow

Application scaling: one shared Python process

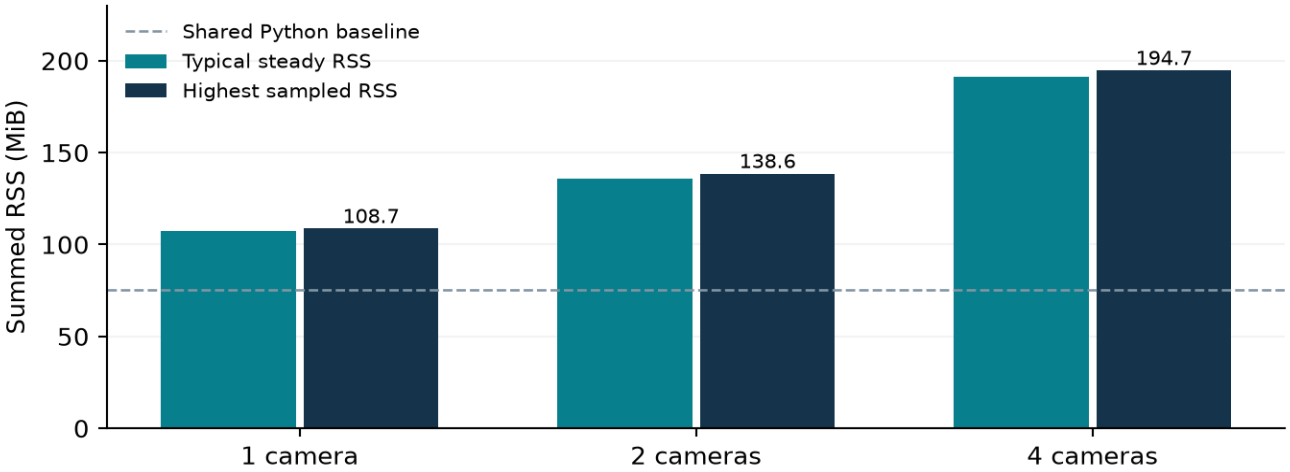


Chart: three 60-second comparison runs per camera count. The longer stability run reached 200.0 MiB; it is included in the headline peak above.

What the measurements establish

Recording adds about **22.2 MiB** and streaming, including decoding, about **10.3 MiB** above the shared Python baseline. Four fast-consumer cameras completed a 10-minute run without sustained queue growth.

Deliberately slow consumers reached **311.1 MiB** and **172 queued frames**. A continuously growing unbounded queue has no fixed memory ceiling.

Scope: H.264, 640 x 360, 25 source FPS, up to 2 selected FPS per camera. No YOLO, VLM or GPU inference. The 250 MiB allowance is for the measured application only, not a complete machine or deployment recommendation.

Production adjustment: Direct camera RTSP connections do not need MediaMTX or mock publishers. Excluding MediaMTX reduces the known-container reference budget from 90.7 to 62.7 MiB (PostgreSQL only). Application-only figures above are unchanged; see page 5.

Workload and method

Setting	Measured configuration
Machine	15.95 GiB physical RAM; Windows-11-10.0.22631-SP0
Runtime	Python 3.12.3; pipeline on Windows, PostgreSQL and MediaMTX in Docker
Video	h264, 640 x 360, 25/1 source FPS, RTSP/TCP
Topology	One shared Python process; recording + streaming workers and a separate unbounded queue per camera; one FFmpeg recording process per camera
Consumer	One counting consumer per camera, with no inference model; stress run adds 0.75 s per frame per consumer
Normal trials	Baseline; recording only; streaming only; full pipeline at 1, 2 and 4 cameras. Three fresh-process repeats per case.
Timing	10 s warm-up + 60 s measurement per normal trial; 10-minute four-camera stability measurement; one slow-consumer trial
Sampling	Application RSS/USS approximately every 0.2 s; host every 1 s; Docker CLI approximately every 5-8 s

Controlled input

Independent connections read the same looping sample-video feed. A separate benchmark publisher normalizes timestamps, uses one-second H.264 keyframes and no B-frames. Existing camera streams and services remain running. This simulates ingestion instances, not four distinct physical camera scenes.

What is counted

The recording path copies compressed video without decoding. Streaming includes PyAV decoding, selection and BGR conversion. The offline recorded-segment decoder is not used. All recording trials write real segments and database metadata.

How statistics are reported

Typical steady memory is the median of the three per-run measurement-window means. Peaks are the highest sampled value across full runs, including startup and draining. Memory held by the sampler itself is included in the measured application; small long-run drift may include retained sample data.

Units and ownership

MiB = 1,048,576 bytes; GiB = 1,024 MiB. RSS is resident process memory; USS is private resident memory. Adding RSS across processes may count shared executable/library pages repeatedly. Workers are threads, so component costs are estimated from isolated-run differences, not exact per-thread ownership.

Components and camera scaling

Application totals: Python plus its recording FFmpeg children. Docker services and the separate benchmark coordinator are excluded from these figures.

Scenario	Cams	Python mean MiB	FFmpeg mean MiB	App mean RSS MiB	App peak RSS MiB	App mean USS MiB
baseline	0	75.1	0.0	75.1	75.4	51.1
recording	1	77.6	19.6	97.3	97.9	55.5
streaming	1	85.4	0.0	85.4	86.1	58.8
full	1	87.9	19.6	107.5	108.7	63.3
full	2	96.7	39.2	136.0	138.6	76.0
full	4	112.6	78.6	191.0	194.7	99.7

Additional memory for one camera

Subtract the 75.1 MiB imported Python baseline. The recording and streaming results are comparative increments; the baseline is shared and counted once.

Component	Additional steady RSS (MiB)	Additional steady USS (MiB)
Recording, including FFmpeg	22.2	4.4
Streaming, including decoding	10.3	7.7
Combined pipeline	32.5	12.2

Scaling estimate and validation

Going from one to two cameras added 28.4 MiB steady summed RSS. Applying that increment predicts **192.8 MiB** for four cameras. The measured four-camera mean was **191.0 MiB**, a prediction error of +0.9%.

Across fast four-camera runs, the highest application RSS was 200.0 MiB. Adding 25% gives a provisional **250 MiB application-only allowance**. Add a deliberate queue budget and future model requirements separately. This does not include the operating system or Docker services.

Four cameras averaged 99.7 MiB private resident memory, versus 191.0 MiB summed RSS. Shared-page accounting is one reason RSS totals exceed unique memory. Whole-host measurements on the next page already include both.

Docker and whole-machine memory

Host used RAM is total physical RAM minus available RAM. It already includes Python, FFmpeg, Docker/WSL, the OS, monitoring and unrelated applications. **Do not add container or process figures to host usage.**

Scenario	Cams	Host used mean GiB	Host available min GiB	WSL RSS mean MiB	Docker Desktop RSS mean MiB
baseline	0	13.97	0.88	498.6	403.8
recording	1	14.04	0.89	501.6	405.6
streaming	1	13.98	1.07	503.8	406.8
full	1	14.14	1.26	504.7	407.8
full	2	14.23	1.12	504.0	410.0
full	4	14.29	1.05	508.8	407.4

Container working memory

Container	Idle mean MiB	4-camera mean MiB	Highest observed MiB
capstone-postgres-dev	55.4	62.7	64.5
mock-cctv	26.8	28.0	29.5

Interpret the layers separately

Docker CLI memory statistics on Linux exclude reclaimable cache. Container values overlap the WSL VM and host totals. WSL figures include all visible vmmem processes, not necessarily Docker alone; Docker Desktop figures sum visible Windows processes and may themselves share pages.

Host usage is not a standalone requirement

The machine averaged 14.29 GiB used during the four-camera comparison windows, with a minimum 1.05 GiB available. Existing Windows applications and VM caching contribute to this load; it cannot all be assigned to the pipeline. Services were left running, so there is no Docker-off or OS-only baseline.

Separate test infrastructure from deployment

The local mock publishers averaged about 83.6 MiB summed RSS. The mock CCTV server and publishers are included in host figures, but real cameras may remove those costs. The Docker VM configuration is a memory cap, not its measured consumption.

Direct-camera production estimate

Production flow: camera RTSP directly to recording and streaming workers. The camera performs encoding and publishing. MediaMTX is optional relay infrastructure and is excluded from this production budget.

The following is a labelled accounting estimate derived from the four-camera trials, not a new direct-camera measurement. The original test results remain unchanged on page 4 and in the raw CSV/JSON data.

Component	Test observation	Production treatment
PostgreSQL container	62.7 MiB mean working memory	Keep as the measured reference
MediaMTX mock relay	28.0 MiB mean working memory	Exclude: 0 MiB for this service
Known-container subtotal	90.7 MiB (database + relay)	About 62.7 MiB (database only)
Local mock-video publishers	83.6 MiB summed RSS	Exclude; real cameras publish their own streams
Recording FFmpeg processes	Included in application figures	Keep; these record incoming video on the server
Docker runtime / VM	Included in measured host usage	Keep an allowance while production services use Docker

What is reduced

Remove MediaMTX's approximately **28.0 MiB** mean working memory from the known-container budget. The two measured containers previously totalled 90.7 MiB; the remaining PostgreSQL reference is 62.7 MiB. This subtotal does not include Docker/VM overhead or future model-serving containers.

What does not change

The measured 200.0 MiB application peak and the provisional 250 MiB application allowance already exclude MediaMTX and the mock publishers. Do not subtract their memory from those figures again. Server-side recording FFmpeg and the streaming decoder are still required.

Why the host total is not simply reduced

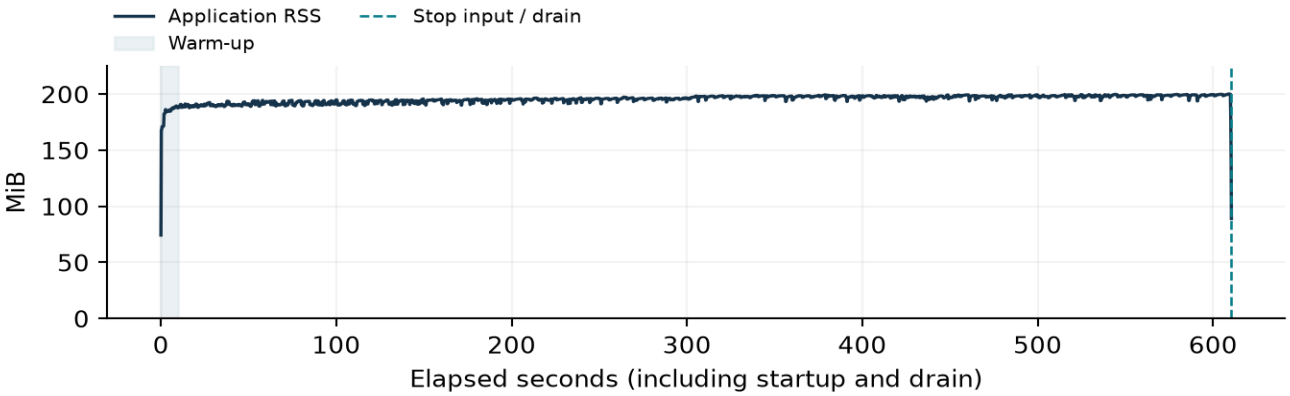
Container working memory, process RSS and WSL allocation are different measurements with shared and cached memory. Removing the relay does not prove an identical physical host-RAM saving. Docker is still needed for PostgreSQL, so its whole VM/desktop overhead cannot be removed. Historical host totals remain as measured.

Remaining verification

Benchmark the intended direct camera URLs, actual resolution/codec and deployed container layout to measure real savings. The production estimate here excludes mock-only services, but still needs OS/runtime, model, queue and operating-headroom budgets.

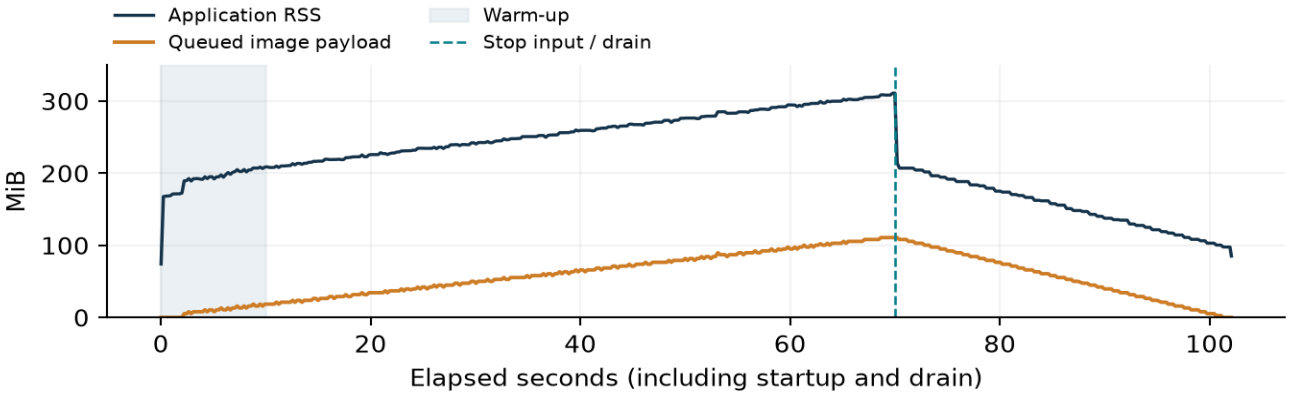
Stability and queue backlogs

Four cameras, fast consumers: 10-minute measurement



All 4,692 selected frames were consumed. Peak RSS was 200.0 MiB; at most 4 frames were queued. Late queue-image growth was 0.00 MiB/min. Late RSS growth was 0.26 MiB/min and includes profiling overhead; this is not a proof of leak-free behavior indefinitely.

Four cameras, slow consumers: 0.75 s per frame each



Stress outcome	Measured value
Peak application RSS	311.1 MiB
Peak backlog	172 frames / 113.4 MiB image data
Late queued-image growth	93.6 MiB per minute
Run time including drain	102.1 s for 10 s warm-up + 60 s measurement

A single 640 x 360 BGR image uses 0.659 MiB before metadata and decoder buffers. If production consistently outruns consumption, an unbounded queue has no fixed memory ceiling. A completed finite stress run does not establish safety for indefinite operation.

Limits, findings and reproduction

Model and resolution limits

These are 640 x 360 ingestion results, not 1080p/4K hardware sizing. Queued image payload grows with pixel count, but total decoding memory cannot reliably be scaled by that factor alone. YOLO/VLM weights, inference workspaces and GPU memory are absent and may dominate the eventual budget.

Consumer and deployment topology

The experiment uses a separate counting consumer per camera. One future shared YOLO consumer must sustain up to 8 selected FPS for four cameras (roughly 125 ms per frame on average including other work). That model and queue arrangement must be benchmarked. The pipeline currently runs on Windows while services run in Docker; a fully containerized/Linux deployment also needs measurement.

Sampling and background activity

Sampled peaks may miss shorter spikes. The in-process sampler retains rows until each run ends, contributing small memory growth. Background applications, source encoding complexity, shared library pages and Docker/WSL caches affect RAM. Three repeats reduce run-to-run uncertainty but do not establish a universal capacity guarantee.

Input-timing finding: excluded initial attempt

A previous four-camera attempt against the existing mock feed stopped after 5.6 seconds with `ValueError: Missing or invalid timing for frame 19`. Its memory results were excluded. All reported comparisons use a separate timestamp-normalized feed. This is not a production fix; investigate that input-timing failure separately.

Reproduce the suite

With the database migrated and the mock RTSP server running:

```
npm run benchmark:memory:suite
```

The suite starts and stops its own publisher, records real video/metadata, runs three repeats per case, performs a 10-minute stability measurement and a slow-consumer test. Existing streams and containers are left running. Allow approximately 35 minutes.

Artifacts and provenance

Completed scenarios: 20. Raw run directory:

`recordings/benchmarks/suite_20261114T145207Z_de86ff/`

Each case contains process.csv, host.csv, docker.csv, summary.json and worker.log. The run directory also contains results.json, environment.json, aggregate.csv and the Markdown report. The source summary is reports/memory-benchmark-2026-11-14.md; the companion table is reports/memory-benchmark-2026-11-14.csv.